

public interface List extends java.util.List {
java.util.Collection
•What are the key features of an abstract class? 抽象类的主要特性是什么？
Cannot be instantiated 不能被实例化；Abstract methods抽象方法；Concrete Methods具体方法；Instance Variable and Constructor实例变量和构造函数
•Can an abstract class be instantiated? 抽象类可以被实例化吗？
False, 抽象类不能被实例化。在Java中，抽象类被设计为其他类的基类，以扩展和提供其抽象方法的特定实现。
•Abstract methods can be defined within a class to save implementation efforts. (T or F) 抽象方法可以在类中定义，以节省实现工作
True
•What are the key issues of implementation when we remove an element from an ArrayList? 当我们从数组列表中删除一个元素时，实现的关键问题是什么？
Shifting elements移动元素；index management修改后续元素的index；Concurrent modification并发修改
•What are the key issues of implementation when we add an element from an ArrayList? 当我们从数组列表中添加元素时，实现的关键问题是什么？
Dynamic array resizing, Automated time complexity, index management

Cost
•Implementing ArrayList: cost of adding and removing
•Cost of operations and measuring efficiency
•ArrayList: : remove(O(n)), add(O(n)), contains(O(n)), size(O(1)), iteration(O(n))
•ArraySet: : contains(O(n)), remove(O(n)), add(O(n)), insert(O(n))
•Cost of Binary Search O(1)–O(logn)
•Cost of SortedArraySet with Binary Search: add(O(n)), remove(O(n)), contains(O(logn)), size(O(1)), iteration(O(n))
问题：
•remove() is compulsory in Iterator implementation. (T or F) 在Iterator实现中，remove()是强制的。(T或F)
False, iterator中只有hasNext和next是强制的
•How does ArrayList make use of the 'type parameter' in its implementation? ArrayList如何在其实现中使用“类型参数”？
类型参数

•Which element will be removed by ArrayList.remove()? 哪个元素将被ArrayList.remove()删除？
remove(int index)删除指定的index上的元素，remove(Object o)删除第一个与提供的o相等的元素
•What does ArrayList.next() check before returning the next element in the list? 在返回列表中的下一个元素之前，ArrayList.next()检查什么？
检查是否有下一个元素调用hasNext检查是否有下一个元素；检查索引是否在范文内当前的cursor是否小于size，如果大于size则代表已经遍历完了
•How does ArrayList.remove() ensure only 1 element can be removed after each call to next()? ArrayList.remove()如何确保每次调用next()后只能删除1个元素？
cursor：指向下一个元素的索引。
lastRet：指向最后一个返回的元素的索引。如果没有这样的元素，则为-1。
remove()方法的实现确保只有在调用next()方法后才能调用remove()方法，并且在一次next()调用后只能调用一次remove()方法
•What can happen if 2 or more iterators running concurrently under the same ArrayList? Name 2 scenarios. 如果两个或多个迭代器在同一个数组列表下并发运行，会发生什么？命名2个场景。
ConcurrentModificationException 异常：ArrayList的迭代器在检测到集合在迭代过程中被修改时，会抛出ConcurrentModificationException 异常。这种情况常见于一个迭代器在迭代的同时，另一个迭代器或线程修改了ArrayList。
数据不一致或意外行为：在没有适当同步的情况下，多个迭代器或线程对同一个ArrayList，进行并发操作，可能会导致数据不一致或产生意外行为。这种情况尤其可能在某些迭代器进行修改操作（如 add、remove）时发生。
•O(log(n)) < O(sqrt(n)) (T or F)
True
•O(n^m) < O(n!) (T or F)
False
•O(2^n) < O(n^n) (T or F)
True
•When analysing the cost of an algorithm, loop usually is the focus. (T or F) 在分析算法的成本时，循环通常是重点。
True
•Which of the following operations is more expensive? 以下哪项操作的成本更高？
•Reading a line from a file
•Reading a line from a user
•Worst case cost analysis is usually more difficult than average cost analysis. (T or F) 最坏情况成本分析通常比平均成本分析更难。
False !!!!!!!!!!!!!!!!!!!!!!! 有可能相同

Recursion
•recursive functions
•factorial function
•fibonacci function
•recursion vs iteration
问题：
•What is the key step in designing a recursive function? 设计递归函数的关键步骤是什么？
•确定递归结束条件（Base Case）。
•定义递归关系（Recursive Case）。
•分解问题（Decompose the Problem）。
•组合结果（Combine the Results）。
•Every recursive function can be rewritten as an iterative function. (T or F) 每个递归函数都可以重写为迭代函数。
•True

Linked Structure
•Testing collection implementations
•Queues
•Motivation for linked lists
•Linked structures for implementing Collections
•A Stack using a Linked List with a header
•A Queue using a Linked List with a header
问题：
•When do you write test cases for “black box” testing? Before or after implementation? 什么时候为“黑盒”测试编写测试用例？实施前还是实施后？
•黑盒测试（Black Box Testing）是一种软件测试方法。在这种方法中，测试人员不需要了解系统的内部实现细节、结构或代码。相反，测试人员关注的是系统的输入和输出，以及系统的功能是否符合预期。
•在软件开发的环境中，通常认为在实现之前为“黑盒”测试编写测试用例是最佳实践。这种方法进行测试驱动开发(TDD)方法的一部分，它强调在编写实际代码之前编写测试。然而，也存在测试用例可能在实现之后编写的场景，特别是在传统的或不正式的开发过程中。
•Explain why array implementations of queue are slow. 解释为什么队列的数组实现速度较慢。
•因为链表实现的队列都是需要通过链接在头节点和尾节点的指针来移动的，因此每次操作都是O(1)。队列的数组实现可能较慢的原因主要在于在某些操作中涉及到大量的元素移动，以及固定容量数组带来的问题。
•Linked list allows data removal by? 链表允许通过什么删除？
•头节点删除，尾节点删除，中间节点删除
•Define references/pointers. 定义/指针的引用。
•在编程中，引用是一种用于间接访问对象或值的机制。引用本身并不存储对象的实际内容，而是存储对象在内存中的地址或位置，通过这个地址可以访问对象的实际内容。在Java等高级编程语言中，引用用于处理对象和数组。
•指针是一个变量，它存储另一个变量的内存地址，通过指针，可以直接访问和操作内存中的数据。指针在C、C++等低级编程语言中广泛使用，但在Java等高级语言中并不直接使用指针，而是使用引用。
•What is the purpose of garbage collection in memory management? 垃圾收集在内存管理中的目的是什么？
•垃圾收集(GC)是Java、C#等编程语言中使用的一种自动内存管理形式。垃圾收集的主要目的是识别和回收程序不再使用的内存，从而防止内存泄漏并优化可用内存的使用。
Sort
•Binary Search O(logn)
•Sorting

- approaches
- selection sort O(n²); in-place
- insertion sort O(n²) in-place
- bubble sort O(n²) in-place
- analysis
- fast sorts

•Design by Divide and Conquer
•Merge Sort(fast sort) : Best Case Time Complexity: O(nlogn) ; Average Case Time Complexity: O(nlogn) ; Worst Case Time Complexity: O(nlogn)
•QuickSort(fast sort) : Best Case Time Complexity: O(nlogn) ; Average Case Time Complexity: O(nlogn) ; Worst Case Time Complexity: O(n²2)
Tree
•Introduction to Trees : recursion对于树来说非常的重要，如果不适用递归那就需要改用迭代
What are trees? level, depth, leaf nodes; balance(如果是平衡二叉树)
•Binary Tree 二叉树
•General Tree 每个节点可以有任意多个子节点
•Different Types of Tree AVL树，Red-black tree
•Tree Ordering 小的插入左子树，大的插入右子树；Trees and Recursion 类似于链表，只不过每个node都有两个子节点

What are they useful for?
• Tic Tac Toe example 井字游戏示例
• Chess
• Taxonomy Tree
• Decision Tree
问题：
•Tree represents an efficient 1-dimensional data structure. (T or F?) 树表示一种高效的一维数据结构(层次结构)
True
•A leaf node in a tree has no children. (T or F?) 树中的叶节点没有子节点
True
•Binary tree has no ordering upon its sibling nodes. (T or F?) 二叉树对其兄弟节点没有排序。
True
•Name 3 applications for tree. Relationship among recursive calls can be expressed in what type of tree? 为树命名3个应用程序。递归调用之间的关系可以用什么类型的树来表示？
•Hierarchical Data Representation分层数据表示，Database Indexing数据库索引；Expression Parsing表达式解析
Tree Addition
•Maps
•Search lists
•Binary search trees
•Tree traversal 数的遍历类似链表，只不过需要递归
•Preorder
•Inorder
•Postorder
•Balanced Search Trees 如果一直只向一个方向添加节点的话就会退化为链表
•AVL Trees 左负右正（左高-1，右高+1）
•search average(O(log n)), worst case(O(log n)); insert average(O(log n)), worst case(O(log n)); delete average(O(log n)), worst case(O(log n)); space average(O(n)), worst case(O(n))
•Implementing Binary Trees
•Implementing General Trees
Table
•Hash tables insert, search, remove O(1)
•Comparison among various search mechanisms 哈希主要通过计算对应的索引的哈希值来匹配，不需要遍历，相比之下大部分搜索都是O(n)或者O(log n)
•Table size: size为N，通过哈希计算得到的结果应该在0–N–1之间
•Hash function 针对Integer: mid-square method; 针对String: 每次取出固定字节长度的字符串，然后将该块解释为单个长整数值，使用modular operators得结果
•Modular hash function 通过求余数，例如H(k) = k mod m 或者 H(k) = a*k mod m
•Hash function examples: avoid collisions; spreads keys evenly in the array; inexpensive to compute(must be O(1))
•Collisions 在避免碰撞的前提下为O(1); 避免碰撞的方式: linear probing, quadratic probing, double hashing
•Dealing with full table: allocate a larger hash table—rehash each from the smaller into the larger—delete the smaller
Graph
•Basic definitions of graph theory : 由一条边连接的两个顶点是adjacent，这两个顶点也称neighbor，连接顶点的边被称为incident edge。
•Properties of graphs : simple graph每两个点之间都只有一条边相连，并且没有边有loop；有向边中出度是1，入度是-1
•Trial: 经过的边都是不同的，可以形成回路也可以不能；Path: trial的基础上加上经过的点也都是不同且无法形成回路；circuit : 几点等于终点形成回路
Spanning Trees: 没有回路的图，如果图G有n个顶点，那么生成树将有n个顶点和n-1条边
•Digraphs and their applications, network flows

时间复杂度:
ArrayList: 继承List
add(E element): O(1) amortized, O(n) in worst case
remove(int index): O(n) -> 返回被移除的元素
get(int index): O(1) -> 返回提供的索引值对应的元素
contains(Object obj): O(n)
size(): O(1) - 返回ArrayList中元素的个数
isEmpty(): O(1) - 检查ArrayList是否为空 boolean
set(int index, E element): O(1) - 替换指定索引位置的元素 -> 返回被替换的元素
indexOf(Object obj): O(n) - 返回指定元素第一次出现的索引位置
lastIndexOf(Object obj): O(n) - 返回指定元素最后一次出现的索引位置
clear(): O(1) - 清空ArrayList中的所有元素
toArray(): O(n) - 将ArrayList转换为数组
LinkedList: 继承List
add(E element): O(1) -> boolean
remove(int index): O(n) -> 被移除的元素
get(int index): O(n) -> 指定index的值
contains(Object obj): O(n) -> boolean
size(): O(1) -> 元素数量
isEmpty(): O(1) -> boolean
clear(): O(1) -> void
add(int index, E element) O(n)
Remove() O(1)删除列表中的头部元素
Vector:
add(E element): O(1) amortized, O(n) in worst case
remove(int index): O(n)
get(int index): O(1)
contains(Object obj): O(n)
HashSet:
add(E element): O(1) average case, O(n) worst case
remove(Object obj): O(1) average case, O(n) worst case
contains(Object obj): O(1) average case, O(n) worst case
size(): O(1)
isEmpty(): O(1)
clear(): O(1)
TreeSet:
add(E element): O(log n)
remove(Object obj): O(log n)
contains(Object obj): O(log n)
size(): O(1)
isEmpty(): O(1)
clear(): O(1)
HashMap:
put(K key, V value): O(1) average case, O(n) worst case
remove(Object key): O(1) average case, O(n) worst case
get(Object key): O(1) average case, O(n) worst case
containsKey(Object key): O(1) average case, O(n) worst case
size(): O(1)
isEmpty(): O(1)
clear(): O(1)
Stack:
push(E element): O(1) -> 返回被推入的元素
pop(): O(1) -> 返回顶部元素并且删除
peek(): O(1) -> 返回顶部元素但不删除
isEmpty(): O(1)
size(): O(1)
Queue:
enqueue(E element): O(1) -> void
remove()element(): O(1) -> 返回被移除的头部元素。
空报值
peek(): O(1) -> 返回头元素，否则null，不移除
isEmpty(): O(1) -> boolean
size(): O(1)
PriorityQueue:
insert(E element): O(log n)
remove(): O(log n)
peek(): O(1)
isEmpty(): O(1)
Binary Search Tree (BST):
insert(E element): O(log n) average case, O(n) worst case (unbalanced tree)
remove(E element): O(log n) average case, O(n) worst case (unbalanced tree)
search(E element): O(log n) average case, O(n) worst case (unbalanced tree)
Graph (Adjacency List representation):
addVertex(V vertex): O(1)
removeVertex(V vertex): O(|V| + |E|)
addEdge(V source, V destination): O(1)
removeEdge(V source, V destination): O(|V|)
getNeighbors(V vertex): O(1)
Trie (Prefix Tree):
insert(String word): O(k), k is the length of the word
search(String word): O(k), k is the length of the word
startsWith(String prefix): O(k), k is the length of the prefix
VL Tree (平衡二叉搜索树) :
insert(E element): O(log n)
remove(E element): O(log n)
search(E element): O(log n)
Red-Black Tree (红黑树) :
insert(E element): O(log n)
remove(E element): O(log n)
search(E element): O(log n)
Hash Table (哈希表) :
insert(E element): O(1) average case, O(n) worst case
remove(E element): O(1) average case, O(n) worst case
search(E element): O(1) average case, O(n) worst case
put(K key, V value): O(1) amortized, O(n) in worst case
get(K key): O(1)
remove(K key): O(1)
containsKey(K key): O(1)
size(): O(1)
isEmpty(): O(1)
clear(): O(1)
LinkedHashSet:
add(E element): O(1)
remove(E element): O(1)
contains(Object obj): O(1)
size(): O(1)
isEmpty(): O(1)
clear(): O(1)
ArraySet
add() O(n)
Remove() O(n)
Contains O(n)
isEmpty() O(1)
SortedArraySet
Add, remove is not better than SortedList, search is better(因为使用了binary search)
Contains() O(log n)

抽象类(Abstract class)和接口(interfaces):

Java接口可以extends多个接口，但是只能extends接口。实体类和抽象类都不行；
抽象类 abstract class 可以选择性地实现自己，因此包含具体方法和抽象方法 public abstract void name(); 抽象类可以 implements 单个或者多个接口，也能 extends 实体类或者抽象类，没有是实现的抽象方法会被接下类的子类实现。
Java Data Structure Collections: Collections=Bag no structure or order maintained, no access limitation, duplicate allowed; List=order, no access limit; Set no structure or order maintained, no access constrains, only property; Stack and Queue are special List, order, limited access; Map key-value pairs unordered;

Hash Table:

- another kind of Table 另一种类型的表
- O(1) in average for insert, lookup, and remove
- 插入、查找和删除的平均耗时为O(1)
- use an array named T of capacity N
- 使用容量N的数组T
- define a hash function that returns an integer int H(string key)
- 定义一个返回整数int H(string key)的哈希函数
- must return an integer between 0 and N-1
- 必须返回0到N-1之间的整数
- store the key and info at T[H[key]]
- 将键和信息存储在T[H[key]]
- H() must always return the same integer for a given key
- 对于给定的键，H()必须始终返回相同的整数
- Hash = O(1) if no collision 如果没有数据碰撞。则代表哈希复杂度为O(1) hash function is simply the sum of ASCII codes of characters in a name (considered all in lowercase) computed mod N=13.
- 哈希函数简单地是一个名称中字符的ASCII码之和(认为都是小写的)。 计算模型N=13。
- What is the Hash Function:
- A hash function is any well-defined procedure or mathematical function for turning data into an index into an array.
- 哈希函数是任何定义良好的过程或数学函数，用于将数据转换为数组的索引。
- The values returned by a hash function are called hash values or simply hashes.
- 哈希函数返回的值称为哈希值或简称为哈希。
- A hash function H is a transformation that 哈希函数H是一个变换
- takes a variable-size input k and
- 取一个可变大小的输入k和
- returns a fixed-size string (or int), which is called the hash value h (that is, h = H(k))
- 返回一个固定大小的字符串 (或int)，称为哈希值h (即h=H (k))
- In general, a hash function may map several different keys to the same hash value.
- 一般来说，哈希函数可以将几个不同的键映射到相同的哈希值。

Example of a Modular Hash Function

- H(k) = k mod m (or k % m)
- H(k) = k mod m (或k % m)
- message1 = '723416'
- 消息 1 = '723416'
- hash function = modulo 11
- 哈希函数=模11
- Hash value1 = (7+2+3+4+1+6) mod 11 = 1
- 哈希值e1 = (7+2+3+4+1+6) mod 11 = 1
- message2 = 'test' = ASCII '74', '65', '73', '74'
- Message2=test=ASCII'74', '65', '73', '74'
- Hash value2 = (74+65+73+74) mod 11 = 0
- 哈希值2= (74+65+73+74) mod 11=0
- another hash function example: a*k mod m
- 另一个哈希函数示例：a*k mod m
- a good hash function has the following characteristics: 一个好的哈希函数包含如下三个特征
- avoids collisions
- 避免碰撞
- spreads keys evenly in the array
- 在数组中均匀分布键
- inexpensive to compute - must be O(1)
- 计算成本低——必须是O(1)

Hash Functions for Strings

- 1.必须小心覆盖从0到capacity-1的范围：
 - 1.在设计哈希函数时，必须确保生成的哈希值能够均匀分布在从0到(capacity-1)的范围内。这意味着哈希函数需要能够充分利用整个可用的哈希表空间，以避免碰撞（即不同的输入产生相同的哈希值）
- 2.一些不好的选择：
 - 1.求和所有的ASCII码：
 - 1.这种方法只是将字符串中所有字符的ASCII码相加，可能会导致大量碰撞。例如，字符串"abc"和"cab"的ASCII码之和是相同的。
 - 2.乘以ASCII码。
 - 1.这种方法会导致哈希值变得非常大，并且在计算机中容易溢出，而且相似的字符串可能会产生非常不同的哈希值，不利于均匀分布。
- 3.重要的见解：
 - 1.字母和数字的ASCII码在八进制的0101到0172范围内：
 - 1.字母和数字的ASCII码在八进制表示时，范围从0101到0172。这意味着这些字符的ASCII码都在十进制的65到122之间。
 - 2.因此，所有有用的信息都在最低的6位中：
 - 1.由于这些字符的ASCII码在这个范围内，所以它们的有用信息都可以通过最低的6位（二进制）来表示。这意味着在设计哈希函数时，可以集中处理这些低位，以有效地利用字符的所有信息。

Hash Functions for Integer Keys

- 1.中平方法（Mid-square method）：
 - 1.这是一个哈希函数设计方法。
- 2.首先对键值进行平方，然后取出结果的中间r位：
 - 1.具体步骤是：
 - 1.对键值（key value）进行平方运算。
 - 2.从平方结果中取出中间的r位（二进制位）。
 - 3.这些中间的r位构成哈希值，该哈希值在范围0到2^r-1内
- 3.这样做效果很好，因为键值的大多数或所有位都对结果有贡献：
 - 1.这个方法的优点在于：
 - 1.充分利用了键值的所有位信息：平方运算将键值的所有位都混合在一起，生成一个新的数，这样即使原始键值中的小变化也会对方平结果产生显著影响。
 - 2.均匀分布：取中间的r位有助于避免仅仅依赖键值的高位或低位，降低了哈希冲突的概率。
 - 3.简单且有效：该方法计算简单，且能提供较好的哈希值分布

Dealing with Collisions 处理碰撞

- open addressing - collision resolution
- 开放寻址冲突解决
- key/value pairs are stored in array slots
- 键/值对存储在数组槽中
- linear probing 线性探测
 - hash(k, i) = (hash1(k) + i) mod N
 - increment hash value by a constant, 1, until free slot is found
 - 得哈希值增加一个常量1，直到找到空槽
 - simplest to implement
 - 最容易实现
 - leads to primary clustering
 - 导致“主集群”
- quadratic probing 二次探测
 - hash(k, i) = (hash1(k) + c1 *i + c2 *i^2) mod N
 - leads to secondary clustering
 - 导致二次聚类
- double hashing
 - hash(k, i) = (hash1(k) + i*hash2(k)) mod N
 - avoids clustering
 - 避免聚类

在哈希表中，每一个表中的空位都可以再额外延申出一组SearchList，也就是说如果有两个不同的数据得到了相同的哈希值，那么可以将相同的内容以List的形式存放在同一个array slot中。这就代表着哈希表永远都不会存在真正意义上的full

Dealing with A Full Table

- allocate a larger hash table
- 分配更大的哈希表
- rehash each from the smaller into the larger
- 从小的重新散列到大的
- delete the smaller
- 删除较小的

Graph

- Two vertices are adjacent if they are joined by an edge.
- 如果两个顶点由一条边连接，则它们是相邻的。
- Adjacent vertices are said to be neighbors.
- 相邻的顶点被称为邻居。
- The edge which joins vertices is said to be incident to them.
- 连接顶点的边被称为“事件边”。
- Two or more edges joining the same pair of vertices are multiple edges.
- 连接同一对顶点的两条或多条边称为多条边。
- An edge joining a vertex to itself is called a loop.
- 连接顶点和自身的边称为循环。
- A graph containing no multiple edges or loops is called a simple graph
- 不包含多条边或环路的图称为简单图

有向图:

A digraph is a directed graph, a graph where instead of edges we have directed edges with arrows (arcs)

indicating the direction of flow.

数图是一种定向图，在这种图中，我们用表示流动方向的箭头（弧）来代替替边

The sum of the values of the degrees, d(V), over all the vertices of a simple graph is twice the number of edges:

一个简单图的所有顶点的度数的和，d(V)是边数的两倍

子图：

If a subgraph H of G spans all of the vertices of G, i.e. V(H) = V(G), then H is called a spanning subgraph of G.

如果G的子图H跨越G的所有顶点，即V (H) =V (G)，则H称为G的生成子图

不连通图:

A disconnected graph is said to be made up of components

一个不连通的图被称为由组件组成

Tree知识:

- 非线性结构: Tree Graph
- AVL例中insertation/removal需要更多的时间，search比较快; 最坏的情况是O(logn)
- A tree is a connected graph with no cycles
- 树是一个没有环的连通图
- A forest is a graph with no cycles and it may or may not be connected
- 森林是一个没有循环的图，它可以连接也可以不连接
- If a tree T has at least two vertices then it has the following properties:
- 如果树T至少有两个顶点，那么它具有以下属性:
- There is exactly one path from any vertex V in T to any other vertex Vj
- 从T中的任何顶点V到任何其他顶点Vj都有一条路径
- The graph obtained from tree T by removing any edge has two components, each of which is a tree
- 从树T中去掉任意一条边得到的图有两个分量，每个分量都是一个树
- |E| = |V| - 1
- A spanning tree of a graph G is
- 图G的生成树是
 - a tree T
 - a spanning subgraph of G.
 - That is, T has the same vertex set as G.

2022 Final

- 最小堆结构
- 1.) Find the index for the element to be deleted
- 2.) Take out the last element from the last level of the heap and replace the indexed element with this element;
- 3.) If the replaced element is greater than any of its child nodes, swap the element with its smallest child;
- 4.) rerepeat 3 to 4 until the node reaches its correct position
- 5.) Output updated binary heap
- 2022 Resit
- 实现bag abstract data type的remove操作
- 1.) User enters target values to remove
- 2.) Loop to check each element in the collection against target value
- 3.) If element not found upon reaching end of collection, return false
- 4.) Otherwise, copy last element onto target element's location and reduce collection's index by 1;
- 5.) Repeat step 2 to 5 for remaining target values.